

Unit VI - Queue



Concept of Queues

- A queue is a common example of a linear list or an ordered list where data can be inserted at and deleted from different ends.
- The end at which data is inserted is called the rear and that from which it is deleted is called the front.
- These limits guarantee that the data is processed in the sequence in which they are entered.
- In short, a queue is a first in first out (FIFO) or last in last out (LILO) structure.
- Consider an ordered list $L = \{a_1, a_2, a_3, a_4, \dots, a_n\}$.
- If we assume that L represents a queue, then a_1 is the front-end element and a_n is the rear-end element. In addition, a_i is behind a_{i-1} .



- Queues are one of the most common data processing structures.
- They are frequently used in most system software such as operating systems, network and database implementations, and other areas.
- Queues are very useful in time-sharing and distributed computer systems where many widely distributed users share the system simultaneously.
- Whenever a user places a request, the operating system adds the request at the end of the queue of jobs waiting to be executed. The CPU executes the job at the front of the queue.



Queue AS ABSTRACT DATA TYPE

- To realize a queue as an abstract data type (ADT), we need a suitable data structure for storing the elements in the queue and the functions operating on it.
- The basic operations performed on the queue include **adding** and **deleting** an element, **traversing** the queue, checking whether the queue is **full or empty**, and **finding** who is at the front and who is at the rear ends.
- A minimal set of operations on a queue is as follows:
 1. **create()**—creates an empty queue, Q
 2. **add(i,Q)**—adds the element i to the rear end of the queue, Q and returns the new queue
 3. **delete(Q)**—takes out an element from the front end of the queue and returns the resulting queue
 4. **getFront(Q)**—returns the element that is at the front position of the queue
 5. **Is_Empty(Q)**—returns true if the queue is empty; otherwise returns false



```

class queue(element)
  declare create() → queue
  add(element, queue) → queue
  delete(queue) → queue
  getFront(queue) → queue
  Is_Empty(queue) → Boolean;
  For all Q ∈ queue, i ∈ element let
    Is_Empty(create()) = true
    Is_Empty(add(i,Q)) = false
    delete(create()) = error
    delete(add(i,Q)) =
      if Is_Empty(Q) then create
      else add(i, delete(Q))
  getFront(create) = error
  getFront(add(i, Q)) =
    if Is_Empty(Q) then i
    else getFront(Q)
  end
end queue

```



Representation of Queue using Sequential organization

Let us see the implementation of the various operations on the queue using arrays.

- **Create** This operation should create an empty queue. Here max is the maximum initial size that is defined.

```
#define max 50
```

```
int Queue[max];
```

```
int Front = Rear = -1;
```



- **Is_Empty** – This operation checks whether the queue is empty or not. This is confirmed by comparing the values of Front and Rear. If $\text{Front} = \text{Rear}$, then **Is_Empty** returns true, else returns false.

```
bool Is_Empty()  
{  
    if(Front == Rear)  
        return 1;  
    else  
        return 0;  
}
```



- **Is_Full** In the definition of the queue ADT, the function for checking the Queue_Full condition is not included.
- When we go in for an array implementation, due to its fixed size, we need to check the state of the queue for being full.
- It is recommended that before we delete an element from the queue, we must check whether the queue is empty or not.
- Similarly, before insertion, the queue must be checked for the Queue_Full state.
- When Rear points to the last location of the array, it indicates that the queue is full, that is, there is no space to accommodate any more elements.

```
bool Is_Full()
{
    if(Rear == max - 1)
        return 1;
    else
        return 0;
}
```



Add This operation adds an element in the queue if it is not full. As Rear points to the last element of the queue, the new element is added at the $(\text{rear} + 1)^{\text{th}}$ location.

```
void Add(int Element)
{
    if(Is_Full())
        cout << "Error, Queue is full";
    else
        Queue[++Rear] = Element;
}
```



Delete This operation deletes an element from the front of the queue and sets Front to point to the next element. Front can be initialized to one position less than the actual front. We should first increment the value of Front and then remove the element.

```
int Delete()
{
    if(Is_Empty())
        cout << "Sorry, queue is Empty";
    else
        return(Queue[++Front]);
}
```



getFront - The operation getFront returns the element at the front, but unlike delete, this does not update the value of Front.

```
int getFront()
{
    if(Is_Empty())
        cout << "Sorry, queue is Empty";
    else
        return(Queue[Front + 1]);
}
```



```
//Queue ADT
class queue
{
    private:
        int Rear, Front;
        int Queue[50];
        int max;
        int Size;
    public:
        queue()
        {
            Size = 0; max = 50;
            Rear = Front = -1 ;
        }
        int Is_Empty();
        int Is_Full();
        void Add(int Element);
        int Delete();
        int getFront();
};
```



```
int queue :: Is_Empty()
{
    if(Front == Rear)
        return 1;
    else
        return 0;
}

int queue :: Is_Full()
{
    if(Rear == max - 1)
        return 1;
    else
        return 0;
}
```

```
void queue :: Add(int Element)
{
    if(!Is_Full())
        Queue[++Rear] = Element;
    Size++;
}

int queue :: Delete()
{
    if(!Is_Empty())
    {
        Size--;
        return(Queue[++Front]);
    }
}

int queue :: getFront()
{
    if(!Is_Empty())
        return(Queue[Front + 1]);
}
```



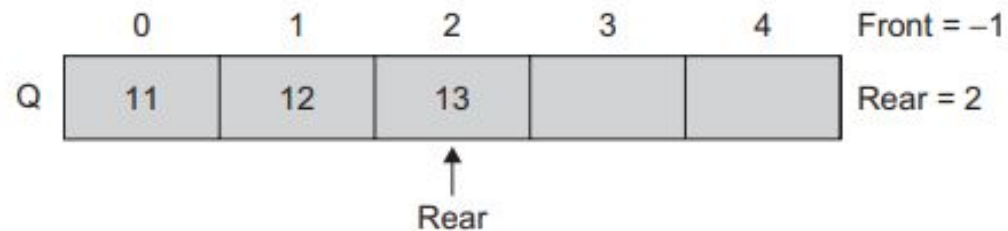
```
void main(void)
{
    queue Q;
    Q.Add(11);
    Q.Add(12);
    Q.Add(13);
    cout << Q.Delete() << endl;
    Q.Add(14);
    cout << Q.Delete() << endl;
    cout << Q.Delete() << endl;
    cout << Q.Delete() << endl;
    Q.Add(15);
    Q.Add(16);
    cout << Q.Delete() << endl;
}
```



Let Q be an empty queue with $\text{Front} = \text{Rear} = -1$. Let $\text{max} = 5$.

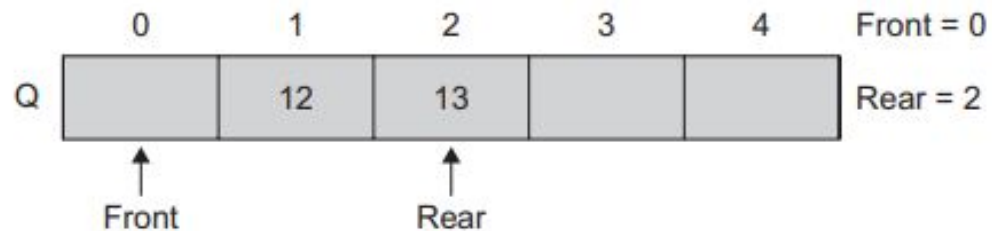


3. $Q.\text{Add}(13)$



4. $A = Q.\text{Delete}()$

Here, $A = Q[+\text{Front}] = Q[0] = 11$



Syntax

The syntax of declaring a queue in C++ is:

```
queue<data_type> variable_name;
```

The header file required to use queue in C++ is:

```
#include <queue>
```



Queue Methods in C++

Method	Description	Time Complexity
<code>empty()</code>	Returns whether the queue is empty. It returns true if the queue is empty; otherwise, false.	$O(1)$
<code>size()</code>	Returns the size of the queue.	$O(1)$
<code>swap()</code>	Exchange the contents of two queues. The queues must be of the same data type, although sizes may differ.	$O(1)$
<code>emplace()</code>	Insert a new element into the queue container. The new element is added to the end of the queue.	$O(1)$
<code>front()</code>	Returns a reference to the first element of the queue.	$O(1)$
<code>back()</code>	Returns a reference to the last element of the queue.	$O(1)$
<code>push(g)</code>	Adds the element 'g' at the end of the queue.	$O(1)$
<code>pop()</code>	Deletes the first element of the queue.	$O(1)$

```
#include <iostream>
#include <queue>
int main() {
    // Create a queue of integers
    queue<int> myQueue;
    // Check if the queue is empty
    if (myQueue.empty()) {
        cout << "Queue is empty." << endl;
    }
    // Push elements into the queue
    myQueue.push(10);
    myQueue.push(20);
    myQueue.push(30);
```



```
// Get the size of the queue
```

```
cout << "Queue size: " << myQueue.size() << endl;
```

```
// Access the front element
```

```
cout << "Front element: " << myQueue.front() << endl;
```

```
// Access the back element
```

```
cout << "Back element: " << myQueue.back() << endl;
```

```
// Pop an element from the queue
```

```
myQueue.pop();
```

```
// Access the new front element
```

```
cout << "Front element after pop: " << myQueue.front() << endl;
```

```
// Swap two queues
```

```
queue<int> anotherQueue;
```

```
anotherQueue.push(100);
```

```
anotherQueue.push(200);
```



```
myQueue.swap(anotherQueue);

cout << "Elements in myQueue after swap:" << endl;
while (!myQueue.empty()) {
    cout << myQueue.front() << " ";
    myQueue.pop();
}
cout << endl;

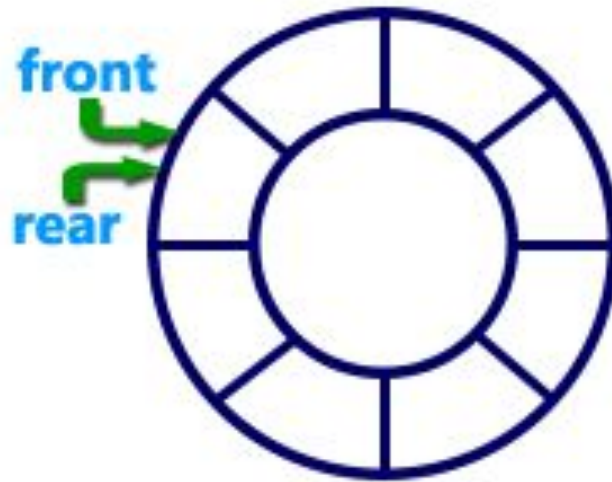
cout << "Elements in anotherQueue after swap:" << endl;
while (!anotherQueue.empty()) {
    cout << anotherQueue.front() << " ";
    anotherQueue.pop();
}
cout << endl;

return 0;
}
```

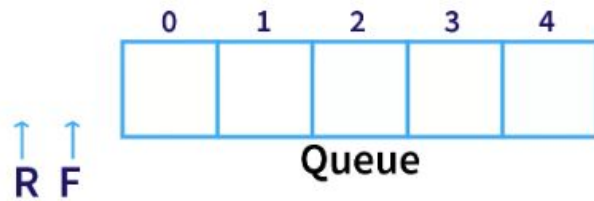


Circular Queue and its advantages

- A circular queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle and the last position is connected back to the first position to make a circle.
- Graphical representation of a circular queue is as follows...

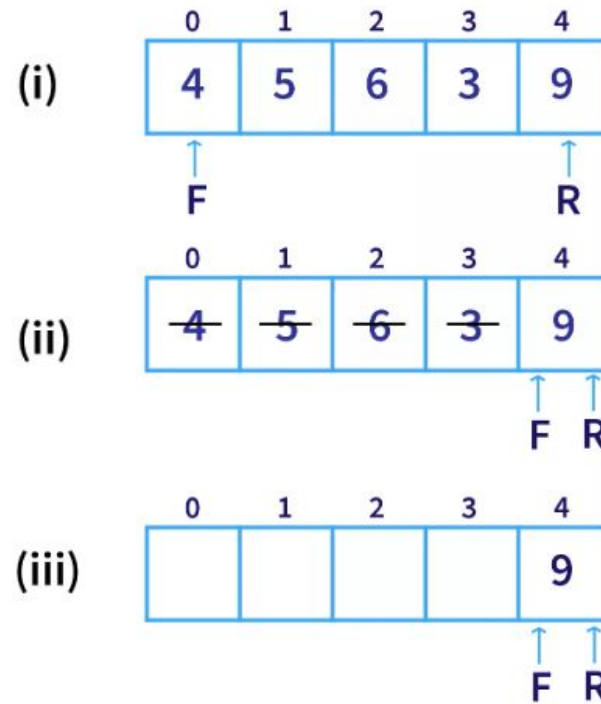


Size of Queue , N=5



Tasks:

- (i) Insert 4, 5, 6, 3, 9.
- (ii) Delete 4, 5, 6, 3.
- (iii) Insert 1.



In this image, in step

(ii), the numbers are strike-through like | 4 | 5 | 6 | 3 | 9 |

(iii) Insertion is not possible as both **F(front pointer)** and **R(rear pointer)** has reached the queue's end. We have space in this simple queue from index 0 to 3 but still cannot insert any new element.



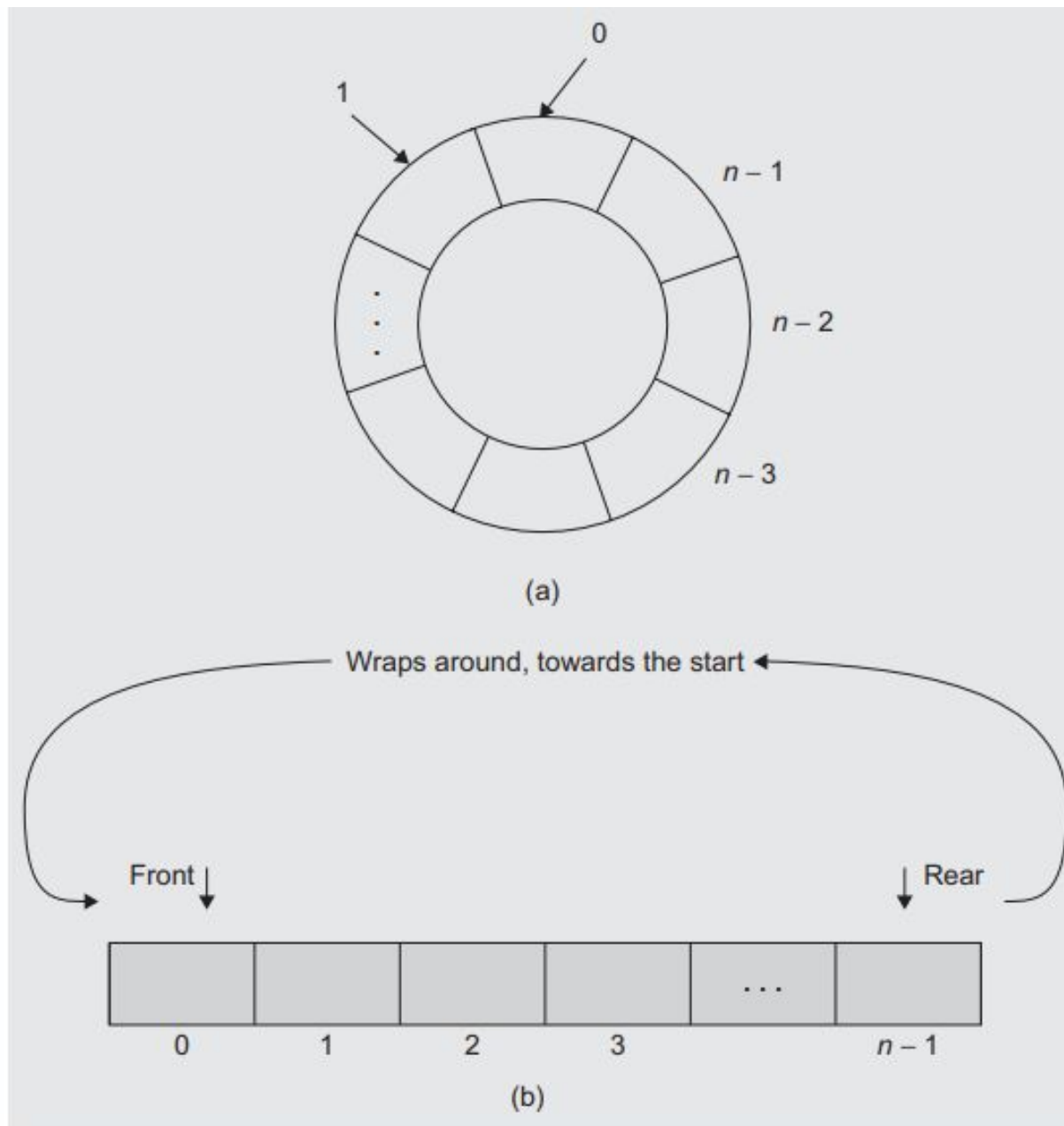
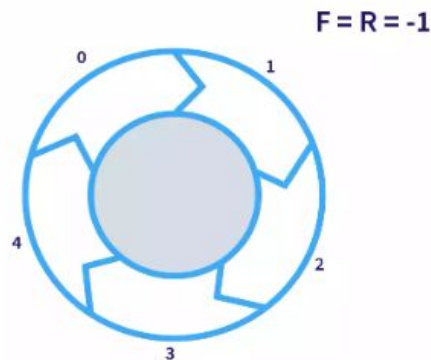


Fig. 5.2 Circular queue (a) Conceptual view (b) Physical view

Size of Circular Queue Q is 5

Initially, Q is empty and the queue pointers

F (front) and R (rear) are initialized to -1

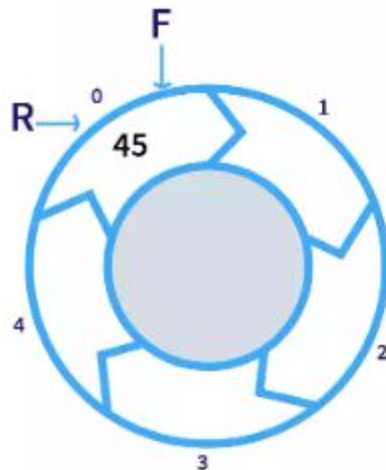


Tasks to perform :-

- (i) Insert 45, 11, 23, 81, 57.
- (ii) Delete 45, 11, 23, 81.
- (iii) Insert 29.
- (iv) Delete 57, 29.
- (v) Insert 17, 19.

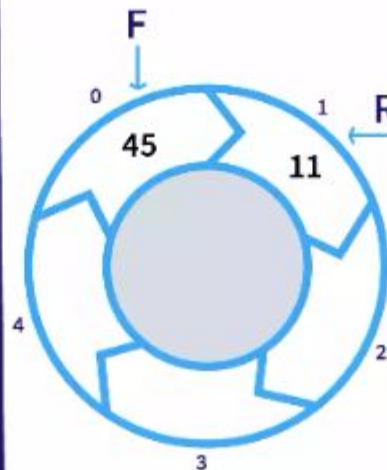
(i) Insert 45:

$F = R = 0$ $Q[R] = 45$



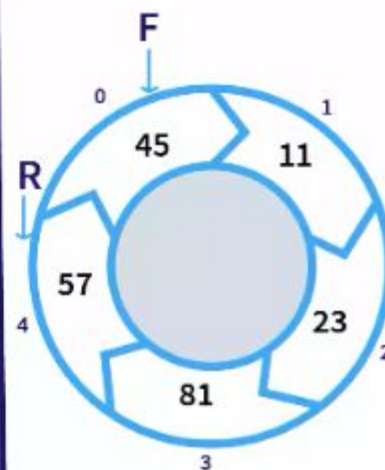
Insert 11:

$R = R + 1$ $Q[R + 1] = 11$



Insert 23, 81, 57:

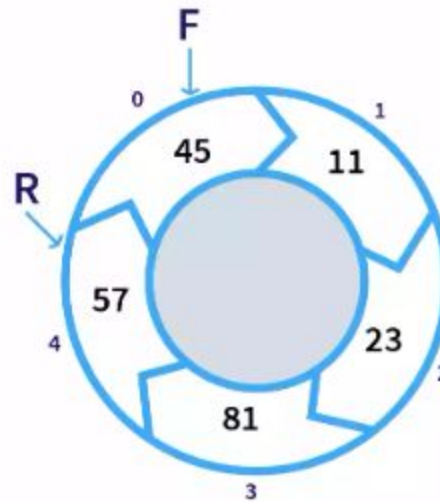
$R = R + 1$ $Q[R + 1] = \text{Val}$



(ii) Insert 32:

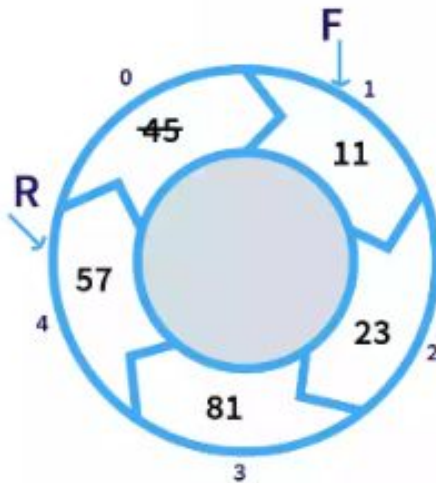
$$(R+1) \% n == F$$

Overflow condition is reached Cannot insert 32



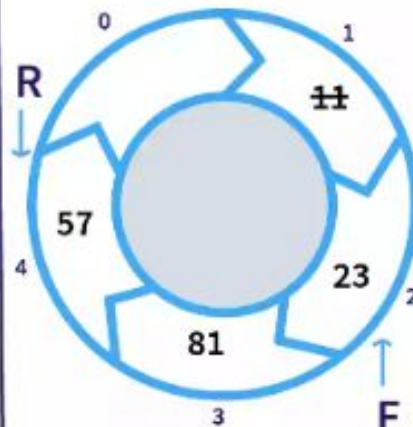
(iii) Delete 45:

$$F = (f+1) \% n$$



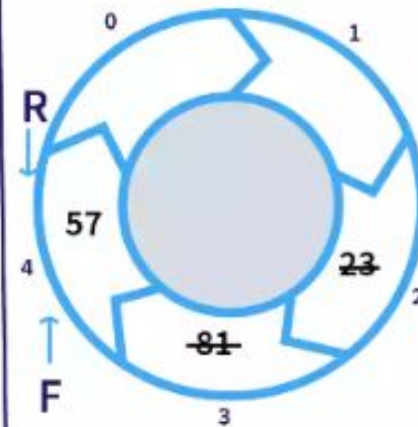
Delete 11:

$$F = (F+1) \% n$$



Delete 23, 81:

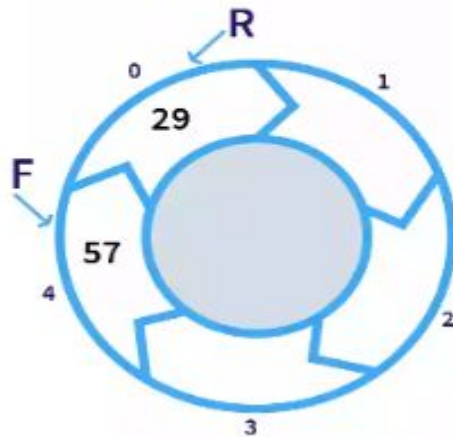
$$F = (F+1) \% n$$



(iv) Insert 29:

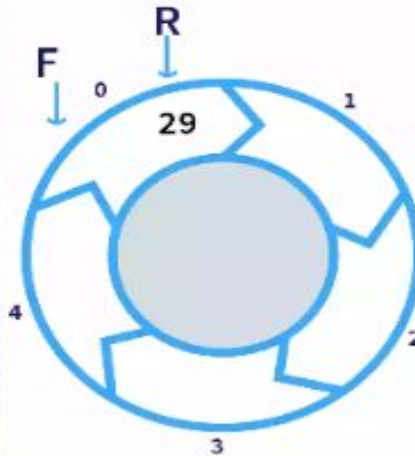
$Q[(R+1)\%n] = 29$

$R = (R+1) \% n$



(v) Delete 57:

$F = (F+1) \% n$

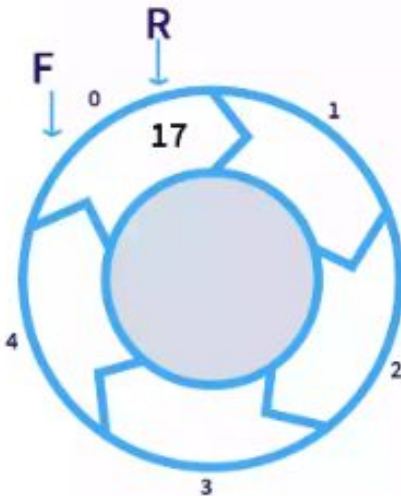


Delete 29:

Queue becomes empty after deleting 29. Any further deletion will lead to underflow condition. The (*) you see below it means that you have to put the image in this screenshot marked with (*) and $F=R=-1$

(vi) Enqueue 57:

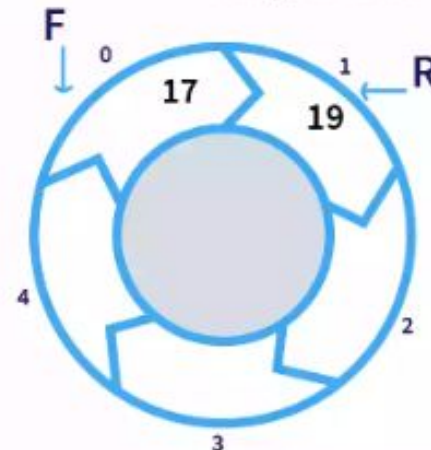
$F = R = 0$



Enqueue 17:

After the last Dequeue, the circular queue becomes empty and insertion starts from 0th position again.

Enqueue 19:



```
#include <iostream>
#include <cstdlib>
#define N 5
int cq[N];
int front = -1;
int rear = -1;
void enqueue(int item);
int dequeue();
int isEmpty();
int isFull();
void display();
```



```
void enqueue(int item)
{
    if (isFull())
    {
        std::cout << "Queue Overflow" << std::endl;
        exit(1);
    }
    if (front == -1)
    {
        front = 0;
    }
    rear = (rear + 1) % N;
    cq[rear] = item;
}
```



```
int dequeue()
{
    int deleted_item;
    if (isEmpty())
    {
        std::cout << "Queue Underflow" << std::endl;
        exit(1);
    }
    deleted_item = cq[front];
    if (front == rear)
    {
        front = -1;
        rear = -1;
    }
    else
    {
        front = (front + 1) % N;
    }
    return deleted_item;
}
```



```
int isEmpty()  
{  
    return (front == -1) ? 1 : 0;  
}
```

```
int isFull()  
{  
    return ((rear + 1) % N == front) ? 1 : 0;  
}
```



```
void display() {
    int i;
    if (isEmpty()) {
        std::cout << "Queue Underflow." << std::endl;
        return;
    }
    i = front;
    if (front < rear) {
        while (i <= rear) {
            std::cout << cq[i] << ",";
            i = i + 1;
        }
    } else {
        while (i != rear) {
            std::cout << cq[i] << ",";
            i = (i + 1) % N;
        }
        std::cout << cq[i] << ",";
    }
    std::cout << std::endl;
}
```



```
int main() {  
    // Insert elements: 42, 66, 35, 59, 17  
    enqueue(42);  
    enqueue(66);  
    enqueue(35);  
    enqueue(59);  
    enqueue(17);  
    display();  
  
    // Delete first 2 elements  
    std::cout << "Deleted Element is: " << dequeue() << std::endl;  
    std::cout << "Deleted Element is: " << dequeue() << std::endl;  
    display();  
  
    // Insert 10  
    enqueue(10);  
    display();  
  
    return 0;  
}
```



Advantages of Using Circular Queues -

The following are the merits of using circular queues:

1. By using circular queues, data shifting is avoided as the front and rear are modified by using the `mod()` function. The `mod()` operation wraps the queue back to its beginning.
2. If the number of elements to be stored in the queue is fixed (i.e., if the queue size is specific), the circular queue is advantageous.
3. Many practical applications such as printer queue, priority queue, and simulations use the circular queue.



Multi-queues

- If more number of queues is required to be implemented, then an efficient data structure to handle multiple queues is required.
- It is possible to utilize all the available spaces in a single array.
- When more than two queues, say n , are represented sequentially, we can divide the available memory $A[\text{size}]$ into n segments and allocate these segments to n queues, one to each.
- For each queue i we shall use $\text{Front}[i]$ and $\text{Rear}[i]$.
- We shall use the condition $\text{Front}[i] = \text{Rear}[i]$ if and only if the i^{th} queue is empty, and the condition $\text{Rear}[i] = \text{Front}[i] + 1$ if and only if the i^{th} queue is full.



- If we want five queues, then we can divide the array $A[100]$ into equal parts of 20 and initialize front and rear for each queue, that is,
 - $\text{Front}[0] = \text{Rear}[0] = 0$ and
 - $\text{Front}[1] = \text{Rear}[1] = 20$, and so on for other queues

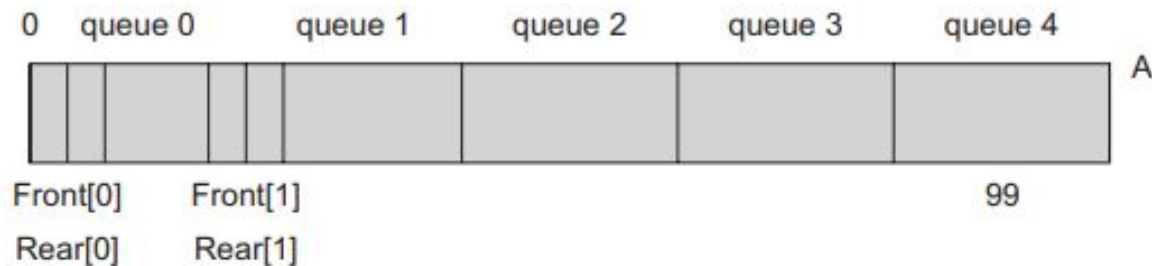


Fig. 5.3 A multi-queue

After adding elements 5 and 8 in the second queue, the resultant queue will be as in Fig. 5.4.

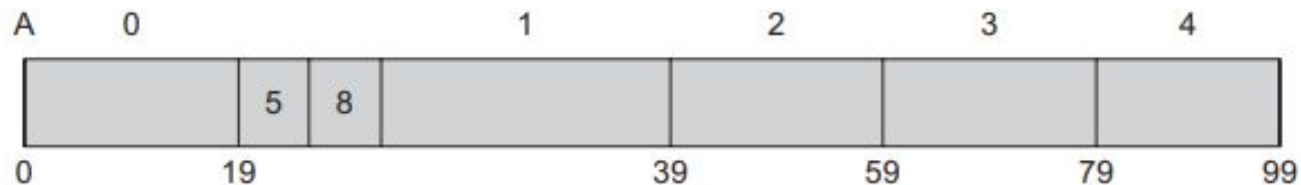
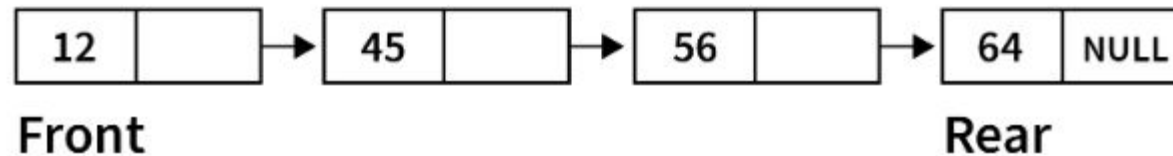


Fig. 5.4 Queue in Fig. 5.3 after addition of elements

Linked Queue

In linked list implementation of a queue, the last inserted node is always pointed by '**rear**' and the first node is always pointed by '**front**'.



Operation on Linked Queue

- Each node of a linked queue consists of two fields: **data and next(storing address of next node)**.
- The data field of each node contains the assigned value, and the next points to the node containing the next item in the queue.
- A linked queue consists of two pointers, i.e., **the front pointer and the rear pointer**. The front pointer stores the address of the first element of the queue, and the rear pointer stores the address of the last element of the queue.
- Insertion is performed at the rear end, whereas deletion is performed at the front end of the queue. If front and rear both points to NULL, it signifies that the queue is empty.



```
struct node {  
    int data;  
    struct node * next;  
};
```

The two main operations performed on the linked queue are:

1. Insertion
2. Deletion



Insertion

Insert operation or insertion on a linked queue adds an element to the end of the queue. The new element which is added becomes the last element of the queue.

Algorithm to perform Insertion on a linked queue:

1. Create a new node pointer.
2. Now, two conditions arise, i.e., either the queue is empty, or the queue contains at least one element.
3. If the queue is empty, then the new node added will be both front and rear, and the next pointer of front and rear will point to NULL.
4. If the queue contains at least one element, then the condition $\text{front} == \text{NULL}$ becomes false. So, make the next pointer of rear point to new node ptr and point the rear pointer to the newly created node ptr



```
#include <iostream>
#include <stdlib.h>

using namespace std;

struct Node {
    int data;
    Node* next;
};

Node* front = nullptr;
Node* rear = nullptr;

int main() {
    Node* head = nullptr;
    insert(head, 10);
    insert(head, 20);

    cout << "front element: " << front->data << endl;
    return 0;
}
```



```
void insert(Node*& ptr, int item)
{
    ptr = new Node;
    if (ptr == nullptr)
    {
        cout << "\nOVERFLOW\n";
        return;
    }
    else
    {
        ptr->data = item;
        if (front == nullptr) {
            front = ptr;
            rear = ptr;
            front->next = nullptr;
            rear->next = nullptr;
        } else {
            rear->next = ptr;
            rear = ptr;
            rear->next = nullptr;
        }
    }
}
```



Deletion

Deletion or delete operation on a linked queue removes the element which was first inserted in the queue, i.e., always the first element of the queue is removed.

Steps to perform Deletion on a linked queue:

1. Check if the queue is empty or not.
2. If the queue is empty, i.e., `front==NULL`, so we just print 'underflow' on the screen and exit.
3. If the queue is not empty, delete the element at which the front pointer is pointing. For deleting a node, copy the node which is pointed by the front pointer into the pointer ptr and make the front pointer point to the front's next node and free the node pointed by the node ptr.



```
#include <iostream>
struct Node {
    int data;
    Node* next;
};
Node* front = nullptr;
Node* rear = nullptr;
void insert(Node*& ptr, int item) {
    ptr = new Node;
    if (ptr == nullptr) {
        std::cout << "\noverflow\n";
        return;
    } else {
        ptr->data = item;
        if (front == nullptr) {
            front = ptr;
            rear = ptr;
            front->next = nullptr;
            rear->next = nullptr;
        } else {
            rear->next = ptr;
            rear = ptr;
            rear->next = nullptr;
        }
    }
}
```



```
void deleteNode(Node*& ptr) {  
    if (front == nullptr) {  
        std::cout << "Underflow";  
        return;  
    } else {  
        ptr = front;  
        front = front->next;  
        delete ptr;  
    }  
}
```



```
int main()
{
    Node* head = nullptr;
    insert(head, 10);
    insert(head, 20);

    std::cout << "front element: " << front->data << std::endl;
    deleteNode(head);
    std::cout << "front element: " << front->data << std::endl;

    return 0;
}
```



```
#include <iostream>

// Structure to create a node with data and the next pointer
struct Node {
    int data;
    Node* next;
};

Node* front = nullptr;
Node* rear = nullptr;

// Enqueue() operation on a queue
void enqueue(int value) {
    Node* ptr = new Node;
    ptr->data = value;
    ptr->next = nullptr;
    if (front == nullptr && rear == nullptr) {
        front = rear = ptr;
    } else {
        rear->next = ptr;
        rear = ptr;
    }
    std::cout << "Node is Inserted\n\n";
}
```



```
// Dequeue() operation on a queue
int dequeue() {
    if (front == nullptr) {
        std::cout << "\nUnderflow\n";
        return -1;
    } else {
        Node* temp = front;
        int temp_data = front->data;
        front = front->next;
        delete temp;
        return temp_data;
    }
}
```



```
// Display all elements of the queue
void display() {
    Node* temp;
    if (front == nullptr && rear == nullptr) {
        std::cout << "\nQueue is Empty\n";
    } else {
        std::cout << "The queue is \n";
        temp = front;
        while (temp) {
            std::cout << temp->data << "--->";
            temp = temp->next;
        }
        std::cout << "NULL\n\n";
    }
}
```



```

int main() {
    int choice, value;
    std::cout << "\nImplementation of Queue using Linked List\n";
    while (choice != 4) {
        std::cout << "1.Enqueue\n2.Dequeue\n3.Display\n4.Exit\n";
        std::cout << "\nEnter your choice : ";
        std::cin >> choice;
        switch (choice) {
            case 1:
                std::cout << "\nEnter the value to insert: ";
                std::cin >> value;
                enqueue(value);
                break;
            case 2:
                std::cout << "Popped element is: " << dequeue() << "\n";
                break;
            case 3:
                display();
                break;
            case 4:
                exit(0);
                break;
            default:
                std::cout << "\nWrong Choice\n";
        }
    }
    return 0;
}

```



Deque

- Deque stands for double-ended queues. Double-ended queues are a type of queue that allows for insertion and deletion at both ends. These dynamically sized sequence containers can be enlarged or contracted on both ends (either from its back or front). The header file to be included is deque.
- **Deque** defines a data structure where elements can be added or deleted at either the front end or the rear end, but no changes can be made elsewhere in the list.
- Figure 5.5 shows the representation of a deque.

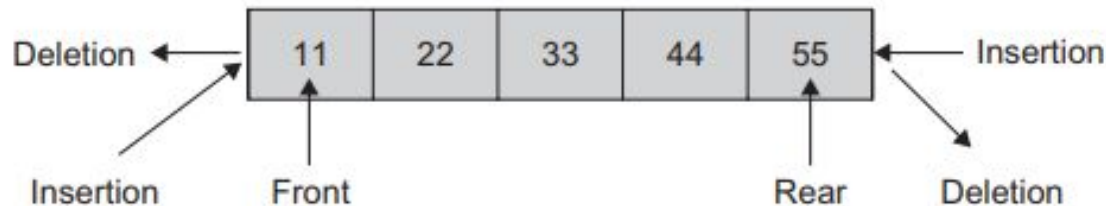


Fig. 5.5 Representation of a deque



The following are the four operations associated with deque:

1. **EnqueueFront()**—adds elements at the front end of the queue
2. **EnqueueRear()**—adds elements at the rear end of the queue
3. **DequeueFront()**—deletes elements from the front end of the queue
4. **DequeueRear()**—deletes elements from the rear end of the queue

For stack implementation using deque, **EnqueueFront** and **DequeueFront** are used as push and pop functions, respectively.



Deque Operation

- Deque allows some of the major unique operations like insertion and deletion from both ends also we can see the front and rear data elements within the constant time complexity.

Basic Syntax

The basic syntax of deque data structure is as follows:

```
deque <data_type> dq_name
```



Push Front /EnqueueFront() -

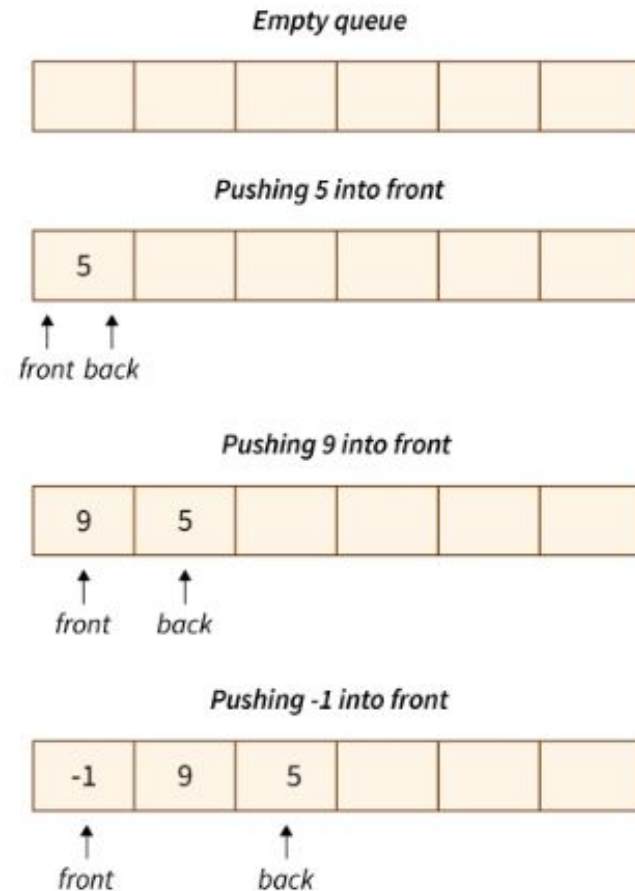
This operation adds the element to the deque from the front end.

```
#include <iostream>
#include <deque>
using namespace std;

void print(deque<int> &dq)
{
    for(auto i: dq)
        cout<<i<<" ";
    cout << endl;
}

int main(){
    deque<int> dq;
    dq.push_front(5);
    dq.push_front(9);
    dq.push_front(-1);
    dq.pop_front();

    print(dq);
}
```



Applications of deque –

- Deque is useful where the data to be stored has to be ordered, compact storage is needed, and the retrieval of data elements has to be faster.

Variations of deque -

- We can have two variations of a deque: **the input-restricted deque and the output-restricted deque.**
- The output-restricted deque allows deletions from only one end and the input-restricted deque allows insertions only at one end.
- The functions to operate an output-restricted deque could be as follows:
DequeFront()(or DequeueRear()), EnqueueFront(), and EnqueueRear()
- The functions to operate an input-restricted deque are as follows: **DequeFront(), DequeueRear(), and EnqueueFront()(or EnqueueRear())**



Pop Front /DequeFront()

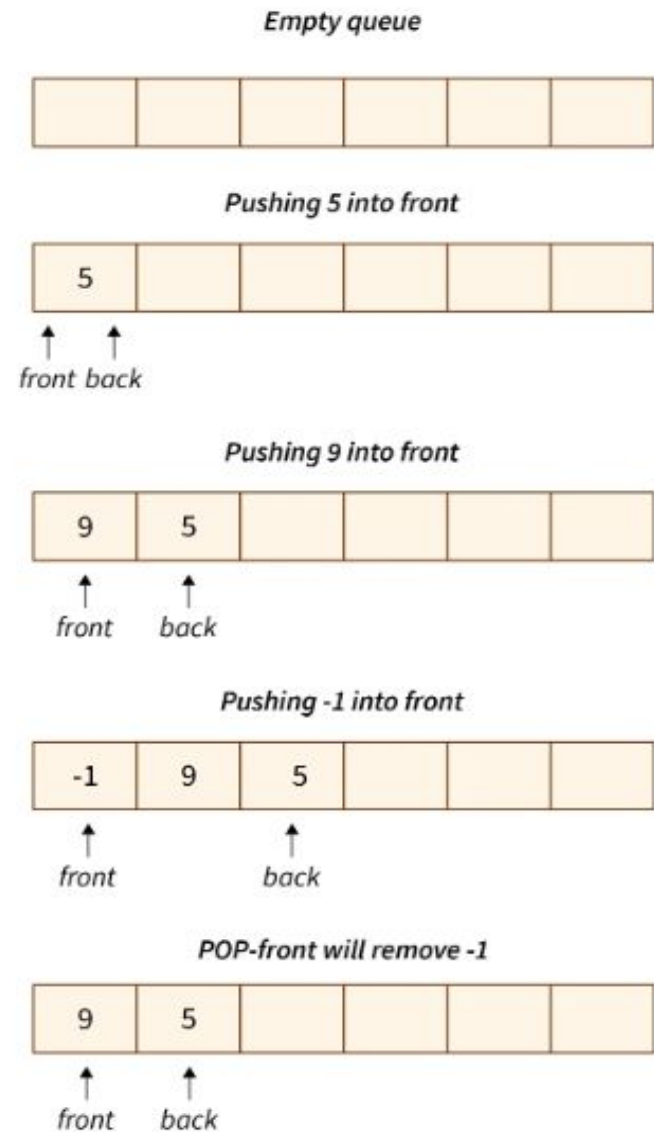
This operation removes the first element from the front end.

```
#include <iostream>
#include <deque>
using namespace std;

void print(deque<int> &dq)
{
    for(auto i: dq)
        cout<<i<<" ";
    cout << endl;
}

int main(){
    deque<int> dq;
    dq.push_front(5);
    dq.push_front(9);
    dq.push_front(-1);
    dq.pop_front();

    print(dq);
}
```



Push Rear /EnqueueRear() -

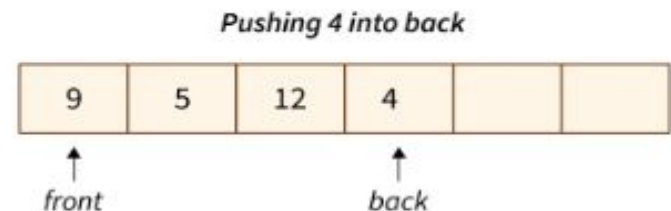
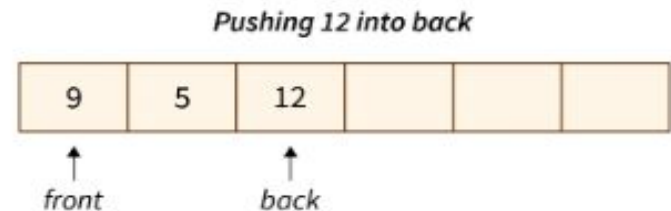
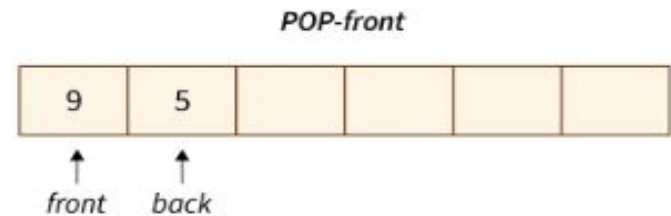
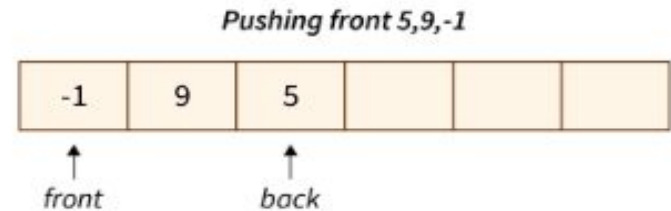
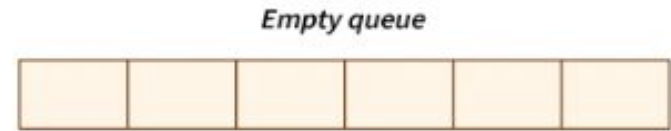
Similar to the last ones, this operation adds the element from the rear/back end.

```
#include <iostream>
#include <deque>
using namespace std;

void print(deque<int> &dq)
{
    for(auto i: dq)
        cout<<i<<" ";
    cout << endl;
}

int main(){
    deque<int> dq;
    dq.push_front(5);
    dq.push_front(9);
    dq.push_front(-1);
    dq.pop_front();
    dq.push_back(12);
    dq.push_back(4);

    print(dq);
}
```



Pop Rear /DequeRear()

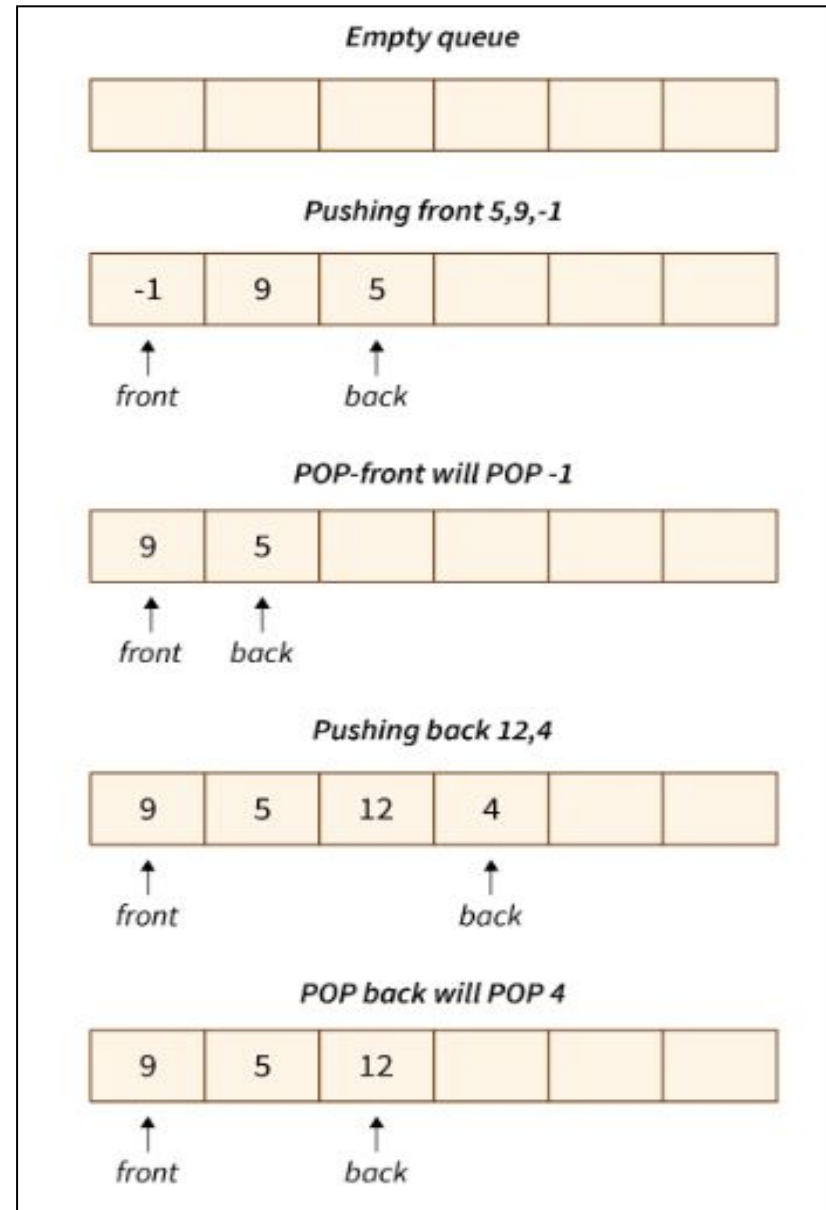
This operation removes the first element from the rear end.

```
#include <iostream>
#include <deque>
using namespace std;

void print(deque<int> &dq)
{
    for(auto i: dq)
        cout<<i<<" ";
    cout << endl;
}

int main(){
    deque<int> dq;
    dq.push_front(5);
    dq.push_front(9);
    dq.push_front(-1);
    dq.pop_front();
    dq.push_back(12);
    dq.push_back(4);
    dq.pop_back();

    print(dq);
}
```



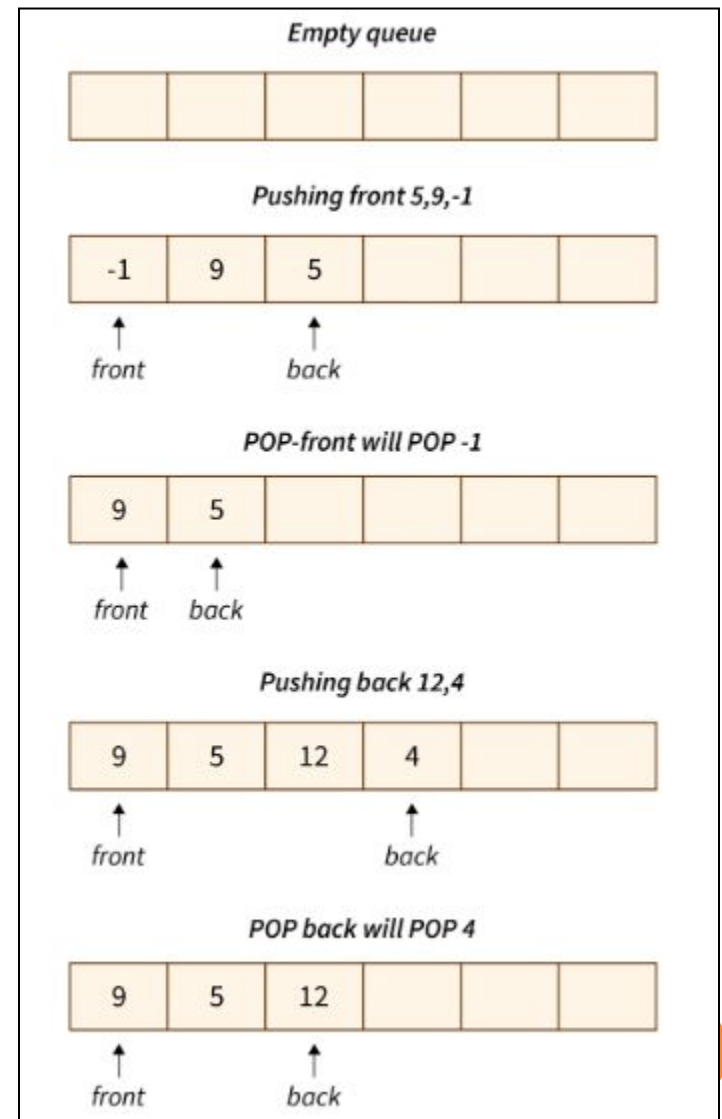
Get Front

This operation fetches the first element from front side.

```
#include <iostream>
#include <deque>
using namespace std;

int main(){
    deque<int> dq;
    dq.push_front(5);
    dq.push_front(9);
    dq.push_front(-1);
    dq.pop_front();
    dq.push_back(12);
    dq.push_back(4);
    dq.pop_back();

    cout << dq.front();
}
```



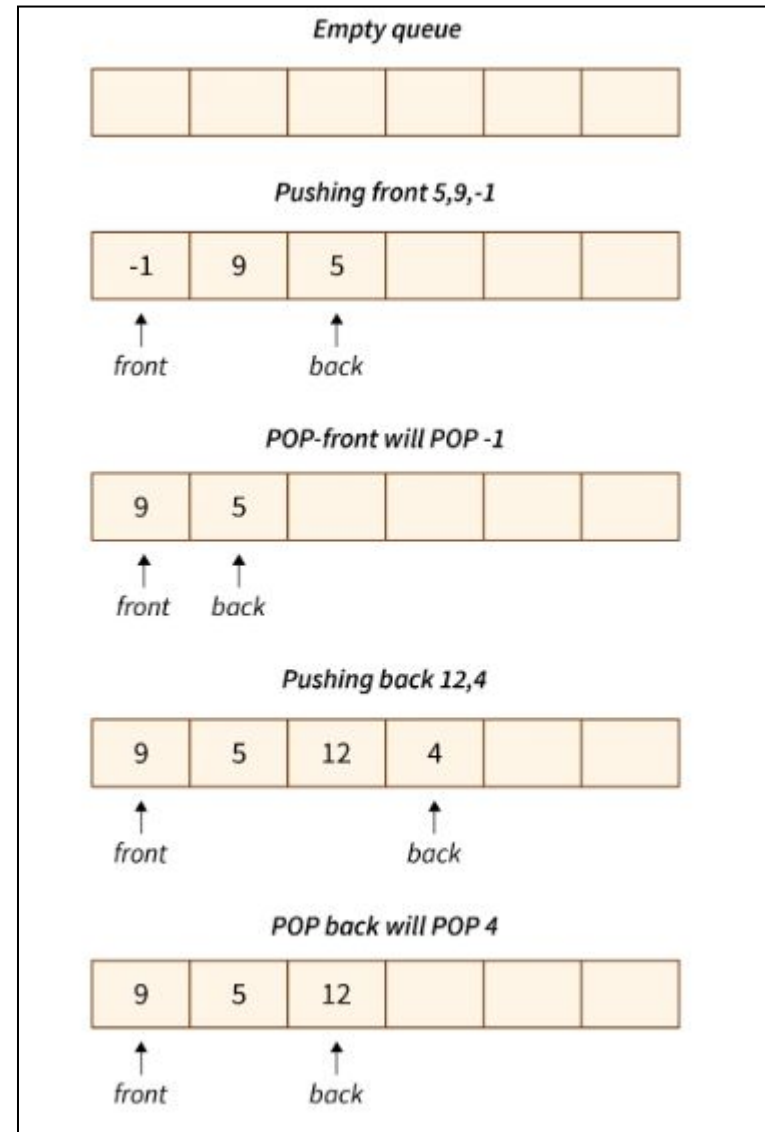
Get Rear

This operation provides the first element from rear side.

```
#include <iostream>
#include <deque>
using namespace std;

int main(){
    deque<int> dq;
    dq.push_front(5);
    dq.push_front(9);
    dq.push_front(-1);
    dq.pop_front();
    dq.push_back(12);
    dq.push_back(4);
    dq.pop_back();

    cout << dq.back();
}
```



```
// Demonstration of deque STL
```

```
#include <iostream>
```

```
#include <deque>
```

```
using namespace std;
```

```
// To print contents of deque
```

```
void print (deque < int >dq)
```

```
{  
    for (auto i:dq)  
        cout << i << " ";  
    cout << endl;  
}
```

```
int main ()
```

```
{  
    // Declaring deque as dq  
    deque < int >dq;
```

```
    //1 is pushed from front
```

```
    dq.push_front (1);
```

```
    //2 is pushed from back
```

```
    dq.push_back (2);
```

```
    cout << "The deque dq is : ";
```

```
    print (dq);
```

```
    //3 is pushed from front
```

```
    dq.push_front (3);
```

```
    cout << "The deque dq is : ";
```

```
    print (dq);
```

```
    //4 is pushed from back
```

```
    dq.push_back (4);
```

```
    cout << "The deque dq is : ";
```

```
    print (dq);
```

```
    //5 is pushed from front
```

```
    dq.push_back (5);
```

```
    cout << "The deque dq is : ";
```

```
    print (dq);
```



```
//maximum size of deque
```

```
cout << "dq.max_size() : " << dq.max_size () << endl;
```

```
//current size of deque
```

```
cout << "dq.size() : " << dq.size () << endl;
```

```
//element at front
```

```
cout << "dq.front() : " << dq.front () << endl;
```

```
//element at back
```

```
cout << "dq.back() : " << dq.back () << endl;
```

```
//element at index 3
```

```
cout << "dq.at(3) : " << dq.at (3) << endl;
```

```
cout << "dq.pop_back() : ";
```

```
dq.pop_back ();
```

```
print (dq);
```

```
cout << "dq.pop_front() : ";
```

```
dq.pop_front ();
```

```
print (dq);
```

```
return 0;
```

```
}
```



Priority Queue- Basic concept, types (Ascending and Descending).

- A priority queue is a collection of a finite number of prioritized elements.
- Priority queues are those in which we can insert or delete elements from any position based on some fundamental ordering of the elements.
- Elements can be inserted in any order in a priority queue, but when an element is removed from the priority queue, it is always the one with the highest priority.
- In other words a priority queue is a collection of elements where the elements are stored according to their priority levels.
- The order in which the elements should be removed is decided by the priority of the element.



- The following rules are applied to maintain a priority queue:
 1. The element with a higher priority is processed before any element of lower priority.
 2. If there were elements with the same priority, then the element added first in the queue would get processed first.
- Priority queues are a special type of queues in which the elements are ordered on the basis of their priority. They are of two types.
- **Max-priority queue:** The first element in the queue is the greatest of all elements. All the elements are in non-increasing order. It is implemented using max-heap. By default, C++ creates a max-priority queue.
- **Min-priority queue:** The first element in the queue is the smallest of all elements. All the elements are in non-decreasing order. It is implemented using min-heap.
- All the methods such as **push()**, **pop()**, **empty()**, **size()**, **swap()**, **emplace()** described above can be applied on priority queues as well.



Syntax

The general syntax of Priority Queue in C++ is:

```
priority_queue<int> variable_name;
```

Syntax of Priority Queue

The syntax of the priority queue(max-heap) is:

```
priority_queue<int> pq;
```

The syntax of the priority queue(min-heap) is:

```
priority_queue<int, vector<int>, greater<int>> pq;
```



```
#include<iostream>
#include<queue>           // Header-file for queue.
using namespace std;

int main() {

    priority_queue < int > p; // Variable declaration.
    p.push(5); // Inserting 5 in a queue, top=5.
    p.push(15); // Inserting 15 in a queue, top=15.
    p.push(10); // Inserting 10 in a queue, top=15.

    cout <<"Number of elements available in 'p' :"<< p.size()<< endl;

    while (!p.empty())
    {
        cout << p.top() << endl;
        p.pop();
    }

    return 0;
}
```



- The `priority_queue` container is implemented as a max heap by default, but it can also be used to create a min heap by passing a custom comparison function to the constructor.
- **Syntax:**

```
priority_queue <int, vector<int>, greater<int>> gq;
```



```
#include <queue>

using namespace std;

int main() {
    priority_queue<int, vector<int>, greater<int>> min_heap;

    // Insert some elements into the min heap.
    min_heap.push(10);
    min_heap.push(5);
    min_heap.push(20);

    // Get the smallest element from the min heap.
    int smallest_element = min_heap.top();

    // Print the smallest element.
    cout << smallest_element << endl; // Output: 5

    return 0;
}
```



Methods of Priority Queue in C++

Method	Definition
<code>empty()</code>	Returns whether the queue is empty.
<code>size()</code>	Returns the size of the queue.
<code>top()</code>	Returns a reference to the topmost element of the queue.
<code>push()</code>	Adds the element 'g' at the end of the queue.
<code>pop()</code>	Deletes the first element of the queue.
<code>swap()</code>	Used to swap the contents of two queues if they are of the same kind; however, their sizes may vary.
<code>emplace()</code>	Used to insert a new element into the priority queue container.

